

Hypothesis Testing Framework

Design and Implementation of Iterative Scientific Method Systems

WAFB Research Team

2026

Introduction to Hypothesis Testing Systems

The Problem Statement

The goal is to create a testing framework and user interface where artificial intelligence can solve hypotheses iteratively over time using the scientific method. The system should:

System Requirements

The framework must support:

Existing Scientific Method Infrastructure

Architecture Overview

The system architecture consists of three main components:

Investigation Methodology

Before designing new components, a systematic investigation was conducted to understand what infrastructure already existed. This investigation followed a structured approach:

Scientific Method Tool Structure

The existing scientific method tool (scientific_method_tool/) provides a complete foundation:

Core Components

Experiment Result Structure

The ExperimentResult dataclass contains: class ExperimentResult: experiment_id: str hypothesis_verified: Optional[bool] confidence: float data_summary: Dict[str, Any] state_changes: Dict[str, Any] metadata: Dict[str, Any]

Analysis System Limitations

The existing ExperimentAnalyzer.analyze_iteration_results() method provides basic aggregation:

This basic analysis is insufficient for determining true consensus, necessitating the design of a more sophisticated consensus engine.

Consensus Algorithm Design

FastAPI Server Infrastructure

The existing FastAPI server (`src/waft/api/main.py`) provides:

The Need for Consensus

Simple majority voting is insufficient for scientific hypothesis testing. A more sophisticated approach is needed that:

Weighted Confidence Consensus Algorithm

Core Principles

The weighted confidence consensus algorithm is based on three principles:

Algorithm Specification

The consensus calculation proceeds as follows: Where the weight for each experiment is its own confidence value. This gives more weight to high-confidence experiments.

Algorithm Implementation

```
def calculate_consensus( experiments: List[ExperimentResult] ) ->
ConsensusResult: """Calculate consensus from experiment results.""" # Minimum
requirement if len(experiments) < 3: return
ConsensusResult( verdict="INCONCLUSIVE", confidence=0.0,
experiment_count=len(experiments) # Weighted confidence calculation total_weight
= sum(e.confidence for e in experiments) weighted_sum = sum( e.confidence *
e.confidence for e in experiments weighted_confidence = ( weighted_sum /
total_weight if total_weight > 0 else 0.0 # Count verdicts verified = sum( 1 for e in
experiments if e.hypothesis_verified is True refuted = sum( 1 for e in experiments if
e.hypothesis_verified is False total = len(experiments) # Consensus criteria if
verified / total >= 0.7 and weighted_confidence > 0.75: return
ConsensusResult( verdict="VERIFIED", confidence=weighted_confidence,
experiment_count=total elif refuted / total >= 0.7 and weighted_confidence > 0.75:
return ConsensusResult( verdict="REFUTED", confidence=weighted_confidence,
experiment_count=total return ConsensusResult( verdict="INCONCLUSIVE",
confidence=weighted_confidence, experiment_count=total
```

System Architecture

Why Weighted Confidence?

The weighted confidence approach is superior to simple averaging because:

Component Overview

The system consists of three main layers:

Backend Components

Consensus Engine

Experiment Runner

API Server

Frontend Architecture

Electron App Structure

React Components

Implementation Details

Data Flow

The data flows through the system as follows:

Halt Mechanism

The halt mechanism allows users to stop experiments gracefully:

WebSocket Protocol

The WebSocket connection provides real-time updates:

```
"type": "experiment_started",  
"experiment_id": "exp_abc123",  
"hypothesis": "...",
```

"timestamp": "2026-01-14T22:30:00Z"

"type": "experiment_completed",

"experiment_id": "exp_abc123",

"result": {

"hypothesis_verified": true,

"confidence": 0.82

"timestamp": "2026-01-14T22:35:00Z"

"type": "consensus_updated",

"consensus": {

"verdict": "VERIFIED",

"confidence": 0.85,

"experiment_count": 5

"timestamp": "2026-01-14T22:40:00Z"

Real-Time Visualization

State Management

The system maintains state at multiple levels:

UI Layout

The Electron UI is organized hierarchically:

Consensus Visualization

When consensus is achieved:

Testing and Validation

Progress Indicators

Each experiment loop displays:

Testing Scenarios

The system should be tested with various scenarios:

Conclusion and Future Work

Validation Criteria

The system is considered successful when:

Summary

This textbook has documented the design and implementation of a hypothesis testing framework that enables iterative scientific method execution with real-time visualization. Key contributions include:

Future Enhancements

Potential future improvements include:

Lessons Learned

The investigation and design process revealed several important insights: